

 QT-AI (큐티 AI 앱) — 코드 품질 게이트 v2.0

문서 버전: v2.0 작성일: 2026-05-08 / 2026-05-14 (v2.0 — Spring Modulith + ArchUnit 도메인 경계 검증 추가) 연관 문서: [DECISIONS](#) / [AGENTS](#) / [docs/adr/0015-pr-validation-spring-modulith](#) / [07_테스트 전략](#) / [14_팀_협업_규칙_Git_브랜치 전략](#) owner: 강태오 (Lead) 목적: PR 머지 전 코드 품질을 자동으로 검증. "빌드는 됐는데 동작 안 함" + "도메인 경계 깨짐" 상황 차단.

 변경 이력

버전	날짜	작성자	주요 변경
v1.0	2026-05-08	강태오 (Lead)	초기 작성
v2.0	2026-05-14	강태오	\$1.5 도메인 경계 검증 신설 — ADR-0015 결정 반영. Spring Modulith 메인 + ArchUnit 보조로 ADR-0001 도메인 경계 절대 규칙을 빌드 단계에서 자동 강제. PR 위반 시 자동 머지 차단.

목차

1. 품질 게이트 전체 구성

2. [Kotlin-Spring Boot 정적 분석](#)

3. [테스트 커버리지 기준](#)

4. [Flutter 품질 게이트](#)

5. [OpenAPI Spectral lint](#)

6. [보안 시크릿 스캔](#)

7. [GitHub Actions CI 전체 구성](#)

1. 품질 게이트 전체 구성

PR 생성

|

|— 1. Spectral lint (OpenAPI 스펙)

|— 2. Kotlin compile + ktlint

|— 3. Spring Boot unit test

|— 4. Test coverage ≥ 60% (도메인 ≥ 80%)

|— 5. ★ Spring Modulith 도메인 경계 검증 (v2.0 신설)

|— 6. ★ ArchUnit 보조 룰 (v2.0 신설)

|— 7. flutter analyze 0 issue

|— 8. flutter test (coverage ≥ 50%)

|— 9. 시크릿 스캔 (TruffleHog)

모두 통과 → PR 머지 허용
하나라도 실패 → PR 머지 차단 + 실패 원인 코멘트

1.5 도메인 경계 검증 (ADR-0015, 2026-05-14 신설)

Modular Monolith v1의 핵심 방어선. ADR-0001 "다른 도메인 패키지 직접 import 금지" 절대 규칙을 빌드 단계에서 자동 강제. 위반 PR은 머지 차단.

1.5.1 도구 구성

도구	역할	검증 방식
Spring Modulith 1.2+	메인 — 모듈 경계 + in-process 이벤트 추적 + PlantUML 다이어그램 자동 생성	@ApplicationModule 메타데이터 기반
ArchUnit 1.3+	보조 — Modulith가 잡지 못하는 미세 룰 (Controller 비즈니스 로직 금지, @Transactional 강제 등)	JUnit5 테스트

1.5.2 도메인 패키지 메타데이터 (필수)

각 도메인 패키지의 package-info.java에 @ApplicationModule 선언:

```
// com/qtai/bible/package-info.java
@ApplicationModule(displayName = "Bible Domain", allowedDependencies = {})
package com.qtai.bible;

import org.springframework.modulith.ApplicationModule;
```

```
// com/qtai/bff/package-info.java
@ApplicationModule(
    displayName = "BFF Aggregator",
    allowedDependencies = { "bible::api", "ai::api", "gatewayauth::api" }
)
package com.qtai.bff;
```

도메인	allowedDependencies (5/14 기준)
gatewayauth	(none) — 외부 OAuth/JWK만
bible	(none)
ai	(none)
journal	(none) — ai.bible 호출은 in-process 이벤트로
simulator	(none) — 본문 좌표만 입력으로 받음

도메인	allowedDependencies (5/14 기준)
bff	bible::api, ai::api, gatewayauth::api

1.5.3 공개 API = @NamedInterface("api")

도메인 간 호출이 허용되는 Facade는 도메인 패키지의 `api/` 하위 패키지에 두고 `@NamedInterface` 선언:

```
// com/qtai/bible/api/package-info.java
@NamedInterface("api")
package com.qtai.bible.api;

import org.springframework.modulith.NamedInterface;
```

다른 도메인은 `com.qtai.bible.api.*`만 import 허용.

`com.qtai.bible.internal.*``com.qtai.bible.repository.*` 직접 import는 자동 거절.

1.5.4 검증 테스트 (필수)

```
// qtai-server/src/test/java/com/qtai/QtaiModulesTest.java
class QtaiModulesTest {
    ApplicationModules modules =
        ApplicationModules.of(QtaiServerApplication.class);

    @Test
    void verifyModuleBoundaries() {
        modules.verify();
    }

    @Test
    void writeDocumentation() {
        new Documenter(modules)
            .writeModulesAsPlantUml()
            .writeIndividualModulesAsPlantUml()
            .writeAggregatingDocument();
    }
}
```

1.5.5 ArchUnit 보조 룰 (W1 초기 5개)

```
// qtai-server/src/test/java/com/qtai/architecture/CodingRulesTest.java
class CodingRulesTest {
    JavaClasses classes = new ClassFileImporter().importPackages("com.qtai");

    @Test void controllers_should_not_call_repositories() {
        noClasses().that().areAnnotatedWith(RestController.class)
            .should().dependOnClassesThat().areAssignableTo(JpaRepository.class)
    }
}
```

```

        .check(classes);
    }

    @Test void entities_should_use_SQLRestriction_not_Where() {
        noClasses().that().areAnnotatedWith(Entity.class)
            .should().beAnnotatedWith("org.hibernate.annotations.Where")
            .check(classes);
    }

    @Test void writes_should_be_transactional() {
        methods().that().areAnnotatedWith(PostMapping.class)
            .or().areAnnotatedWith(PutMapping.class)
            .or().areAnnotatedWith(PatchMapping.class)
            .or().areAnnotatedWith>DeleteMapping.class)
            .should().beAnnotatedWith(Transactional.class)
            .check(classes);
    }

    @Test void no_chromadb_imports() {
        noClasses().should().dependOnClassesThat()
            .resideInAnyPackage("..chromadb..", "io.weaviate..",
"org.elasticsearch..")
            .check(classes);
    }

    @Test void no_kafka_v1() {
        // v1은 Kafka 사용 금지 (ADR-0001-0007)
        noClasses().should().dependOnClassesThat()
            .resideInAnyPackage("org.springframework.kafka..")
            .check(classes);
    }
}

```

1.5.6 build.gradle.kts 의존성

```

dependencies {
    implementation("org.springframework.modulith:spring-modulith-starter-
core:1.2.4")
    testImplementation("org.springframework.modulith:spring-modulith-starter-
test:1.2.4")
    testImplementation("com.tngtech.archunit:archunit-junit5:1.3.0")
}

```

1.5.7 위반 시 PR 코멘트 표준

❌ 도메인 경계 위반 – ADR-0001 도메인 패키지 절대 규칙 위반

com.qtai.bff.controller.PassageController가
com.qtai.bible.repository.BibleVerseRepository를 직접 import했습니다.

해결:

1. com.qtai.bible.api.BibleFacade Interface를 통해 호출하세요.
2. 새 메서드가 필요하면 com.qtai.bible.api에 추가 후 PR 분리하세요.

참조: ADR-0001 / AGENTS.md "도메인 경계 절대 규칙" / 18_코드_품질_게이트.md §1.5

2. Kotlin·Spring Boot 정적 분석

2.1 ktlint (Kotlin 코드 스타일)

```
// build.gradle.kts (root)
plugins {
    id("org.jlleitschuh.gradle.ktlint") version "12.1.0"
}

ktlint {
    version.set("1.2.1")
    android.set(false)
    reporters {
        reporter(ReporterType.CHECKSTYLE)
    }
    filter {
        exclude("**/generated/**")
        exclude("**/*.g.kt")
    }
}
```

```
# 검사
./gradlew ktlintCheck

# 자동 수정
./gradlew ktlintFormat
```

2.2 1차 사고 패턴 자동 감지 (Custom Detekt)

```
// build.gradle.kts
plugins {
    id("io.gitlab.arturbosch.detekt") version "1.23.3"
}

detekt {
    config.setFrom("detekt.yml")
    buildUponDefaultConfig = true
}
```

detekt.yml — 핵심 룰:

```
potential-bugs:
  active: true
  UnsafeCast:
    active: true

style:
  active: true
  ForbiddenComment:
    active: true
    values: ['TODO:', 'FIXME:', 'HACK:'] # W4 이후 금지

complexity:
  active: true
  LongMethod:
    active: true
    threshold: 60 # 60줄 초과 메서드 경고

custom-rules:
  # Controller에 비즈니스 로직 금지
  NoBusinessLogicInController:
    active: true
  # @RestController 클래스 내에서 repository 직접 호출 X
```

2.3 @Transactional 누락 자동 감지

```
# CI에서 실행 - 쓰기 UseCase에 @Transactional 없으면 경고
./gradlew detectMissingTransactional
```

```
// buildSrc/src/main/kotlin/DetectMissingTransactional.kt
// UseCase implement execute() 메서드 중 DB 호출하는데 @Transactional 없는 것 탐지
```

CI에서 경고(warning) — 즉각 머지 차단은 하지 않되 PR 코멘트로 알림.

3. 테스트 커버리지 기준

3.1 커버리지 기준

계층	최소 커버리지	도구
Domain 전체	80%	JaCoCo
Application (UseCase)	70%	JaCoCo

계층	최소 커버리지	도구
Presentation (Controller)	50%	JaCoCo
Infrastructure	40%	JaCoCo
전체 서비스	60%	JaCoCo

```
// build.gradle.kts
tasks.jacocoTestCoverageVerification {
    violationRules {
        rule {
            element = "PACKAGE"
            includes = listOf("com.qtai.*.domain.*")
            limit {
                minimum = "0.80".toBigDecimal() // domain 80%
            }
        }
        rule {
            element = "CLASS"
            excludes = listOf(
                "com.qtai.*.infrastructure.*",
                "com.qtai.*.*Application",
            )
            limit {
                minimum = "0.60".toBigDecimal() // 전체 60%
            }
        }
    }
}
```

3.2 커버리지 확인

```
# 커버리지 리포트 생성
./gradlew test jacocoTestReport

# 리포트 위치
# build/reports/jacoco/test/html/index.html

# 기준 미달 시 빌드 실패
./gradlew jacocoTestCoverageVerification
```

4. Flutter 품질 게이트

4.1 flutter analyze

```
fvm flutter analyze
# 출력 예:
# Analyzing flutter-app...
# No issues found!
```

분석 옵션 (`analysis_options.yaml`):

```
include: package:flutter_lints/flutter.yaml

linter:
  rules:
    - avoid_print          # print() 대신 debugPrint
    - prefer_const_constructors
    - prefer_final_locals
    - require_trailing_commas
    - sort_pub_dependencies
    - avoid_dynamic_calls   # dynamic 타입 호출 경고

analyzer:
  errors:
    invalid_annotation_target: ignore # freezed false positive
  exclude:
    - "**/*.g.dart"
    - "**/*.freezed.dart"
    - "**/build/**"
```

4.2 Flutter 테스트 커버리지

```
# 커버리지 포함 테스트 실행
fvm flutter test --coverage

# 커버리지 리포트 (genhtml 필요)
genhtml coverage/lcov.info -o coverage/html
open coverage/html/index.html
```

기준:

- Widget test: 전체 Widget의 50%+
- UseCase/Repository test: 60%+
- domain (순수 Dart): 80%+

4.3 domain layer 순수성 검증

```
# domain/ 디렉토리에 flutter/dio import 있는지 확인 (08번 § 3.4 원칙)
grep -r "package:flutter\|package:dio\|package:riverpod" \
  flutter-app/lib/features/*/domain/ \
```



```
flutter-app/lib/core/*/domain/  
# 결과 없어야 - 있으면 CI 실패
```

5. OpenAPI Spectral lint

5.1 실행

```
# 전체 스펙 lint  
npx @stoplight/spectral-cli lint \  
  apis/*/openapi.yaml \  
  --ruleset .spectral.yaml \  
  --format junit \  
  --output spectral-results.xml
```

5.2 커스텀 룰 검증 (.spectral.yaml 정합)

```
# 룰셋 자체 문법 검증  
npx @stoplight/spectral-cli lint .spectral.yaml --ruleset .spectral.yaml  
# 오류 없어야
```

PR에서 확인할 것:

- **error** 레벨: 머지 차단
- **warn** 레벨: PR 코멘트 알림 (머지 허용)

6. 보안 시크릿 스캔

6.1 TruffleHog (git history 전체)

```
# PR에서 새 커밋만 스캔  
docker run --rm -v "$(pwd):/repo" \  
  trufflesecurity/trufflehog:latest \  
  git file:///repo \  
  --since-commit HEAD~1 \  
  --only-verified \  
  --fail  
  
# 종료 코드 0 → 시크릿 없음  
# 종료 코드 183 → 시크릿 발견 → CI 실패 + 즉시 강태오 멘션
```

6.2 감지 대상

```
# .trufflehog.yaml
detectors:
  - DeepSeek      # sk-*
  - JWT           # eyJ...
  - MySQL         # 연결 문자열 내 비밀번호
  - GoogleOAuth   # ya29.*, client_secret
  - PrivateKey    # -----BEGIN RSA PRIVATE KEY-----
```

6.3 시크릿 발견 시 대응

```
# git history에서 시크릿 제거 (강태오 즉시 실행)
git filter-branch --force --index-filter \
  'git rm --cached --ignore-unmatch path/to/secret_file' \
  --prune-empty --tag-name-filter cat -- --all

git push origin --force --all
```

⚠ 이미 **push**된 시크릿은 즉시 무효화 필수. DeepSeek key → DeepSeek console에서 rotate. Google OAuth → Google Console에서 재발급.

7. GitHub Actions CI 전체 구성

```
# .github/workflows/ci.yml
name: QT-AI CI

on:
  pull_request:
    branches: [develop, main]
  push:
    branches: [develop]

jobs:
  spectral-lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm install -g @stoplight/spectral-cli
      - run: spectral lint apis/*/openapi.yaml --ruleset .spectral.yaml

  backend-build:
    runs-on: ubuntu-latest
    needs: spectral-lint
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: '21', distribution: 'temurin' }
      - run: ./gradlew build jacocoTestCoverageVerification
```

```

env:
  GRADLE_OPTS: -Xmx2g

flutter-test:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: subosito/flutter-action@v2
      with: { flutter-version: '3.24.5', channel: 'stable' }
    - run: cd flutter-app && flutter pub get
    - run: cd flutter-app && flutter analyze
    - run: cd flutter-app && flutter test --coverage
    - run: |
        grep -r "package:flutter\\|package:dio" flutter-
app/lib/features/*/domain/ && exit 1 || exit 0

secret-scan:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
      with: { fetch-depth: 0 }
    - name: TruffleHog scan
      uses: trufflesecurity/trufflehog@main
      with:
        path: ./
        base: ${{ github.event.pull_request.base.sha }}
        head: ${{ github.event.pull_request.head.sha }}
        extra_args: --only-verified --fail

```

7.1 게이트 통과 기준 요약

게이트	실패 시
Spectral error	머지 차단
Gradle build 실패	머지 차단
Spring Modulith <code>verifyModuleBoundaries()</code> 실패 (v2.0)	머지 차단
ArchUnit 룰 위반 (v2.0)	머지 차단
커버리지 60%/domain 80% 미달	머지 차단
flutter analyze issue	머지 차단
flutter test 실패	머지 차단
domain layer에 flutter 의존	머지 차단
시크릿 발견	머지 차단 + 강태오 즉시 멘션
Spectral warn	PR 코멘트 경고 (머지 허용)

목표: W1부터 CI를 green으로 유지. Red 상태에서 다음 작업 시작 금지 (1차 사고 패턴 재발).